

Event reporting project

Silvia Eremia - SSA

Cristian-Ștefan Sbârnea - MTI

Gabriel Olaru - ABD

Andrei Băroiu - SD

Github: <https://github.com/csbarnea/event-reporting>

Abstract

In modern urban and digital environments, timely and accurate information about events is essential for effective decision-making and rapid response. Events such as traffic incidents, infrastructure failures, public safety issues, or community-reported problems often rely on fragmented reporting channels, leading to delays, data loss, or lack of situational awareness. A unified event reporting system can significantly improve how such information is collected, managed, and disseminated.

With the widespread adoption of smartphones and web technologies, users can act as real-time data sources by reporting events directly from their location. Combining geolocation, multimedia content, and structured metadata enables richer and more reliable event descriptions. At the same time, administrators and decision-makers require centralized tools to visualize events, monitor their evolution, and receive immediate notifications when new reports are submitted.

This project addresses these needs by proposing a cross-platform event reporting application that integrates web and mobile technologies with scalable backend services. By leveraging modern data processing, messaging, and storage technologies, the system aims to provide a flexible and extensible solution capable of supporting various event types and usage scenarios, while ensuring real-time awareness and efficient event management.

Introduction

This project aims to design and implement a simple and scalable event reporting service intended for a wide range of event types, including but not limited to emergencies, public incidents, infrastructure issues, and community-reported events. The system is structured around three main components: a backend server responsible for processing and storing reported events in a centralized database, a web-based client that allows users to both report events through a browser interface and visualize existing reports on an interactive map, and a mobile application for Android devices that enables fast and intuitive event reporting directly from the field.

Each reported event is defined by several key attributes: the time of reporting, geographic location, alert code, textual description, a supporting image, and a classification tag used for

categorization and filtering. This structured data model enables efficient storage, retrieval, and analysis of reported information.

Once an event is submitted, the system automatically sends email notifications to designated system administrators. Administrators are identified by their full name, Romanian mobile phone number, and email address, ensuring timely awareness and response to newly reported events.

The application is implemented using Jupyter and leverages a modern technology stack to support reliability, scalability, and real-time data handling. Apache Kafka is used for event streaming and decoupled communication between components, while Apache Spark enables data processing and analysis. PostgreSQL serves as the persistent storage layer. The backend is developed using Flask, the web interface is built with Next.js, and the mobile application is developed using React Native. Firebase is integrated for auxiliary services such as authentication and media storage. Together, these technologies provide a robust foundation for a cross-platform, event-driven reporting system.

Opinion

Real-Time Mobile Event Reporting as a Tool for Safer Communities

Cristian-Ștefan SBÂRNEA

Abstract

This essay explores the potential of a real-time event reporting mobile system as a tool for making communities safer. The mobile application can be used to report critical information about an event, including its type, a description and the exact location. At the same time, it examines how the use of this application can significantly improve situational awareness and emergency response times. Nevertheless, the success of real-time event reporting platforms hinges on data protection and security measures.

Existing Solutions

The Neighbourhood Safety Alarm System (IJSAT, 2025) [1] and the IoT-based public safety alert system by Zhang et al. (2025) [2] demonstrate how real-time event reporting can contribute to enhancing community safety. For example, Zhang et al. developed a system that uses distributed sensors to achieve rapid detection and alerting for emergencies, managing to achieve an impressive constant latency of under 500ms. The Neighbourhood Safety Alarm System uses GPS and a mobile application to deliver instant notifications and support user incident reporting, encouraging the civic implication. Both systems put emphasis on two crucial aspects of developing an effective event reporting application, notifying users in real-time of close and possibly dangerous incidents, and providing precise information about the exact location of each event. By delivering instant alerts to users in the affected area, these platforms enable individuals to take immediate precautions, such as evacuating or seeking shelter, thereby reducing the risk of harm. At the same time, authorities could

intervene more efficiently, respond faster, and coordinate interventions with higher precision. As a result, communities become better prepared to handle these situations, minimize risks and create safer environments for everyone.

Impact and Results of Real-Time Emergency Response Apps

The effectiveness of real-time emergency response apps has been validated through multiple studies and real-world deployments. According to the IAFC PulsePoint paper [3], the app matches victims of sudden cardiac arrest with nearby CPR-trained bystanders and provides the location of the nearest automated external defibrillator. PulsePoint notifications enable simultaneous alerts to both citizen and emergency responders. The use of the application resulted in a significant reduction in response time by citizens (3.5 versus 5.6 minutes) and a dramatic increase in layperson arrival (70% versus 15%). This fast response can translate to a big improvement in survival rates and a lot of lives saved. A recent review by Tong et Al. (2025) [4] shows that mobile apps for emergency response can make a significant difference in saving lives during sudden cardiac arrest. The study found that people who used these apps were 34% more likely to survive until discharge or 30 days compared to those who relied only on traditional emergency services. The apps also increased the number of bystanders who performed CPR, making it 25% more likely that someone would help before paramedics arrived. Despite these benefits, there are still some challenges. In some places, not enough people use the app, and sometimes notifications are missed due to user settings. Still, the research clearly shows that real-time emergency applications help communities respond faster and save lives. Simultaneously, most of the users found these kinds of applications to be easy to use and reliable.

Security and Data Protection in Real-Time Event Reporting

The framework described by Shaffi (2025) [5], places strong emphasis on both the operational efficiency and the security aspects of real-time incident reporting systems. The framework is designed to ensure that all sensitive incident data is protected from unauthorized access, tampering or breaches. This is achieved through robust encryption protocols and secure authentication mechanisms. The framework also highlights the importance of data integrity, ensuring that incident reports are accurate, unaltered, and traceable throughout their lifecycle. This is particularly crucial in emergency response scenarios, where the reliability of data can directly impact decision-making and also public safety. By maintaining high standards of confidentiality, compliance and integrity, it protects users and strengthens the trustworthiness of real-time reporting platforms.

Conclusion

Real-time event reporting systems represent a transformative approach to how society works, by enhancing community safety and emergency response. By leveraging technologies such as geolocation and mobile applications, these platforms deliver instant notifications about nearby incidents, ensuring that both individuals and authorities receive timely and accurate information. The integration of precise location data enables users to take immediate precautions and allows responders to act swiftly and effectively, minimizing risks and improving outcomes. Studies have shown that such systems significantly increase survival

rates, accelerate emergency interventions and foster greater public engagement in safety efforts. However, the success of these platforms depends on the robust implementation and the security measures that protect user data, maintain privacy, and ensure the integrity of incident reports. By combining real-time communication, accurate geolocation, proven results and strong security, these systems create safer environments and empower communities to respond proactively to emergencies.

A Modern Event-Driven Architecture: Implementing Kafka-to-Spark Data Pipelines for Reporting and Analysis

Silvia Eremia

Abstract

Processing and analyzing large volumes of incident data in real time represents a significant challenge for modern event reporting systems. This essay argues that an event-driven data pipeline built with Apache Kafka and Apache Spark provides an efficient and scalable solution for handling continuous data streams generated by web and mobile applications. By decoupling data ingestion from data analysis, this architecture enables real-time reporting, fault tolerance, and future extensibility. The essay highlights the advantages of Kafka as a messaging backbone and Spark as a distributed processing engine, emphasizing their suitability for incident reporting and analytical workloads.

Introduction

The rapid growth of user-generated data has significantly changed how modern applications are designed and implemented. Event reporting systems, in particular, must handle continuous streams of data originating from web and mobile platforms, often under unpredictable load conditions. Traditional monolithic or synchronous architectures struggle to meet these requirements due to scalability limitations and tight coupling between system components.

In my opinion, an event-driven architecture is the most appropriate solution for such systems. Existing research shows that event-driven designs improve scalability and resilience by decoupling producers from consumers [6]. In this context, Apache Kafka and Apache Spark are commonly used together to support real-time processing of incident data.

Event-Driven Data Ingestion with Apache Kafka

Apache Kafka is widely recognized as a foundational technology for distributed event streaming. It provides a persistent, fault-tolerant log where events can be stored and

consumed asynchronously. According to Kreps et al. (2011)[6], Kafka was specifically designed to handle high-throughput, low-latency data streams in distributed environments.

In an incident reporting application, each reported incident can be modeled as an immutable event containing metadata such as timestamp, location, and type. Publishing these events to Kafka topics ensures reliable data ingestion while allowing consumers to process the data independently. This design avoids direct dependencies between the backend server and analytical components, improving maintainability and system robustness.

Stream Processing and Analysis with Apache Spark

While Kafka ensures reliable data transport, it does not provide advanced analytical capabilities. Apache Spark addresses this limitation by offering a unified engine for large-scale data processing. According to Zaharia et al. (2016) [7], Spark achieves high performance for batch and streaming workloads primarily due to its in-memory computation model.

Using Spark Structured Streaming, incident data can be processed as continuous streams, enabling near real-time aggregation and analysis. For example, Spark can compute incident frequencies per geographic area or identify temporal trends that indicate abnormal situations. This approach is consistent with the Dataflow model proposed by Akidau et al. [8], which emphasizes correctness and low latency in stream processing systems.

In my view, separating analytical processing from the core backend and assigning it to Spark is essential for maintaining system performance, allowing the backend to remain focused on handling client requests efficiently.

Architectural Benefits and Design Considerations

The Kafka–Spark pipeline offers several architectural advantages supported by existing literature. First, it enables loose coupling between system components, a principle strongly advocated in modern data-intensive system design. Second, it supports both real-time analytics and historical reprocessing, as Kafka retains event data for a configurable period.

However, some authors argue that event-driven systems introduce operational complexity due to the need for distributed infrastructure [9]. While this concern is valid, recent advances in containerization and orchestration have significantly reduced deployment and maintenance overhead. In my opinion, the analytical and scalability benefits outweigh these challenges when designing an incident reporting system.

Conclusion

In conclusion, an event-driven data pipeline based on Apache Kafka and Apache Spark represents a robust and scalable solution for incident reporting applications, architecture which we chose for our project. Kafka provides reliable and fault-tolerant data ingestion, while Spark enables efficient real-time and batch data analysis. Supported by existing

research and industry practices, this architecture aligns well with the requirements of large-scale, real-time event reporting systems.

Designing a Lightweight and Scalable REST Backend Using Flask for Real-Time Event Reporting Systems

Gabriel Olaru

Abstract

Real-time event reporting platforms ingest heterogeneous, user-generated data from web and mobile clients. Because ingestion is the boundary where velocity, variety, and veracity can be controlled, its design strongly influences downstream analytical value. This essay argues that a lightweight REST backend can act as a dedicated ingestion gateway by validating and normalizing events, persisting them transactionally, and forwarding clean records to processing pipelines. We summarize limitations of streaming ingestion architectures for user-driven workloads and motivate a pragmatic design using Flask endpoints and PostgreSQL for ACID storage, with hybrid handling of media objects.

1. Introduction

The growth of event-driven applications has increased the importance of ingestion in Big Data ecosystems [12], [13]. In incident reporting systems, events arrive from heterogeneous clients and must be accepted reliably under bursts, while preserving data quality for downstream analytics [12], [13]. Defects introduced at entry propagate through the pipeline and reduce analytical value, so ingestion is an architectural concern, not only a transport step [12], [17].

We position a lightweight REST backend as a controlled ingestion layer that makes boundary trade-offs explicit, including validation strictness, persistence strategy, and integration with downstream systems [17].

2. Big Data ingestion challenges in user-driven event reporting

User-driven reporting combines irregular arrival patterns with heterogeneous content [12]–[14]. Velocity appears as bursts during incidents, requiring low-latency request handling and safe concurrent writes [13], [15]. Variety is intrinsic: structured attributes (timestamps, coordinates, categories) coexist with semi-structured text and unstructured media [12], [14]. Veracity is harder to guarantee because users may submit incomplete or inconsistent input, so early validation and normalization reduce downstream correction cost [13], [4]. Volume typically accumulates over time, which favors incremental scaling and controlled buffering that preserves consistency [15], [17].

3. Existing ingestion architectures and limitations

Streaming ingestion is powerful for continuous machine-generated flows, but user submissions are often interactive, bursty, and schema-variable [13], [14]. When validation is deferred downstream, malformed events can propagate into analytics jobs, increasing the cost of cleanup and reducing trust in results [14], [17]. Distributed stream processing models also emphasize careful handling of time and state for out-of-order data, which can be heavy to

adopt end-to-end when the primary need is trustworthy intake and persistence [15]. A practical pattern is therefore to validate and persist events at the boundary, then publish clean records to internal consumers (for example, via a durable log) [18].

4. A lightweight REST backend as a controlled ingestion layer

A REST backend (e.g., implemented with Flask) provides stable, versioned endpoints to clients and can enforce constraints before records enter the wider pipeline [16], [17]. Stateless request handling supports horizontal scaling during bursts, while explicit error semantics and idempotent behaviors reduce duplicates during client retries [18], [19].

PostgreSQL supports this role through ACID transactions and concurrency control, acting as a reliable buffer before export to downstream processing [12], [17]. To address variety, structured event attributes are stored relationally, while large media objects are stored externally, preserving schema clarity and avoiding database bloat [14], [17]. Once committed, validated events can be appended to a durable log (e.g., Kafka) for multiple downstream consumers, without coupling user traffic directly to streaming infrastructure [18].

This design maps cleanly to Big Data values: velocity via stateless scaling, variety via hybrid storage, veracity via early validation, volume via incremental growth and durable persistence, and value via consistent records that enable reliable aggregation and decision support [13], [17].

Beyond basic validation, the ingestion gateway can enforce lightweight governance and operational controls that protect downstream analytics. For example, it can record provenance metadata such as client type and acceptance time, apply rate limiting during incident bursts, and emit structured logs and metrics for observability. These controls help detect anomalies early and keep quality rules close to the point of entry, reducing the need for corrective processing later in the pipeline [12], [13], [17]. Current Big Data research emphasizes extending these mechanisms with robust schema evolution, late-event handling, and exactly-once processing guarantees to improve reliability at scale [12], [13]. When streaming is required, publishing validated events to a durable log after a successful database commit supports replayable processing without exposing end users directly to streaming semantics [14].

Modeling ingestion as a boundary service reduces coupling between data producers and analytical consumers, enabling downstream pipelines to evolve independently and supporting maintainability and incremental scalability in Big Data systems with evolving requirements [15], [17].

5. Limitations and conclusion

REST ingestion is well suited for human-driven workloads but is not optimized for extreme machine-generated streams, where direct ingestion into specialized messaging systems may offer higher throughput [18]. PostgreSQL provides strong consistency but should not become a long-term analytical store, so exporting data to downstream systems remains necessary as datasets grow [17]. Lightweight stacks also shift operational responsibilities to developers, who must implement monitoring and fault handling explicitly [15], [17].

Despite these trade-offs, a controlled REST backend is a pragmatic ingestion layer for real-time event reporting. It improves data quality at the point of entry and reduces unnecessary operational complexity, complementing Big Data processing components by ensuring clean and well-structured inputs [13], [17].

Implementing Event Reporting Dashboards and User Interfaces with Big Data Architectures

Andrei Băroiu

Introduction

The transition from traditional, centralized disaster management to participatory sensing has revolutionized crisis informatics. In this paradigm, citizens act as decentralized sensors, generating vast quantities of data that exceed the capabilities of legacy web architectures. The proposed Event Reporting System (ERS) aims to monitor critical incidents, such as fires and protests, by addressing the four pillars of Big Data: Volume, Velocity, Variety, and Veracity [19].

Conventional architectures based on monolithic relational databases are prone to failure during large-scale crises due to system overload and high data noise. Consequently, a modern architecture utilizing Apache Kafka for ingestion, Apache Spark for stream processing, and Next.js with WebGL for high-performance visualization is required to ensure both system resilience and data integrity [20].

Architectural Solutions for Big Data Problems

The technical challenges inherent in an ERS can be mapped to specific architectural solutions, as summarized in Table [1](#).

Volume and Velocity: Managing the Scalability Cliff

During major events, data generation shifts from linear to exponential. Relational databases (SQL) often face row-locking and timeouts when attempting to process concurrent writes alongside complex geospatial queries. To mitigate this, Apache Kafka is employed as a high-throughput buffer [21]. Kafka decouples the report submission process from data processing, effectively handling "backpressure" to maintain a responsive user interface even under extreme load.

Mapping Big Data Problems to Architectural Components

Dimension	Challenge	Architectural Solution
Volume	Storage of binary data/logs	S3/MinIO via Kafka Connect
Velocity	Real-time ingestion	Apache Kafka Buffer
Variety	Heterogeneous data types	Spark Structured Streaming
Veracity	Information reliability	Bayesian Confidence Scoring

Variety: Processing Heterogeneous Data

The ERS ingests a mix of structured (GPS coordinates, timestamps) and unstructured data (images, text). Apache Spark Structured Streaming serves as the unified engine to process this variety [22]. By applying Natural Language Processing (NLP) to text descriptions and computer vision models to images, the system converts unstructured data into queryable tags.

Veracity: The Trust Deficit and Truth Discovery

Veracity remains the primary hurdle in crowdsourced reporting. The Truth Discovery module utilizes Bayesian Estimation to calculate a Confidence Score (C) for each report, defined in the equation below:

$$C_{event} = \frac{\sum_{i=1}^n (w_i \cdot \alpha_i)}{\sum_{i=1}^n w_i}$$

Where w_i represents the reliability weight of user i , and α_i is the correlation of their report with existing spatial clusters.

Clustering and Spatial Verification

To prevent "phantom events" generated by misinformation, the system utilizes Density-Based Spatial Clustering of Applications with Noise (DBSCAN). Isolated reports are filtered as noise, while dense clusters trigger verified alerts.

Visualization Strategy

Browser Performance and GPU Acceleration

Rendering thousands of standard markers in the DOM can crash modern browsers. The ERS employs Server-Side Tiling and WebGL-based rendering via Deck.gl. Unlike standard mapping APIs, Deck.gl utilizes the user's GPU to render millions of points smoothly, enabling data-driven styling where visual properties like opacity are tied directly to the veracity score [23].

Architectural Diagram

Data flow from Next.js → Kafka → Spark → Deck.gl

Proposed System Architecture for Real-time Disaster Visualization.

Conclusion

The development of a resilient Event Reporting System requires treating the frontend as a dynamic window into a high-velocity data stream. By adopting a streaming architecture to resolve backend bottlenecks and a WebGL strategy for the interface, the system can scale from a localized tool to a robust city-wide disaster management platform.

Implementation

The event reporting application is implemented as a distributed, event-driven system that integrates web, mobile, and backend components to ensure reliable, real-time data collection, processing, and visualization. The architecture follows a modular and loosely coupled design, allowing each component to operate independently while interacting through RESTful APIs and a message streaming layer. This approach enhances scalability, fault tolerance, and maintainability, supporting simultaneous usage by multiple clients across different platforms.

The backend server is developed using Flask, which exposes RESTful endpoints for event submission, retrieval, updates, and administrative management. Upon submission, the backend performs data validation and normalization, ensuring all required fields—reporting timestamp, precise geographic coordinates, alert code, descriptive text, classification tags, and image references—are correctly formatted before storage. Event metadata is stored in a PostgreSQL database, providing transactional integrity and efficient querying capabilities, while associated media, such as photos, are stored in Firebase Storage, with references linked in the database for retrieval.

To achieve real-time processing and scalability, the system incorporates Apache Kafka as a message broker. Each newly reported event is published as a Kafka message, allowing asynchronous communication between producers (web/mobile clients) and consumers (analytics, notifications, or monitoring services). Apache Spark subscribes to relevant Kafka topics and processes incoming event streams for tasks such as aggregation, filtering, geospatial analysis, and generating summary statistics. This enables administrators and users to perform analytics in near real-time, supporting features like heatmaps, alerts, and event trends.

The web client, built with Next.js, supports two main functions: reporting new events through a browser-based form and visualizing existing events on an interactive, dynamic map. Users can specify the exact location of an event by clicking on the map, or input GPS coordinates manually, ensuring high spatial accuracy. The interface also provides advanced filtering and search functionalities based on parameters such as location, alert type, and classification tags. Map markers dynamically update in real-time as new events are reported.

The mobile application, developed in React Native for Android, is optimized for on-the-go reporting. It automatically captures device GPS coordinates, allows the attachment of images or video, and supports quick textual descriptions. The app communicates securely with the backend using HTTPS, ensuring end-to-end encryption for sensitive location and user data.

Once an event is successfully recorded, the backend triggers automatic notifications via SMS and email to designated administrators, providing immediate situational awareness. The system also maintains detailed logs for auditing and troubleshooting purposes. The entire implementation is orchestrated and documented using the Jupyter platform, which allows interactive testing, visualization of data flows, and rapid iteration during development.

This architecture enables the system to handle high event volumes, support real-time analytics, and provide cross-platform accessibility, making it a robust solution for event reporting in diverse contexts.

Conclusions

The event reporting application demonstrates a robust, scalable, and flexible system for collecting, processing, and visualizing events across web and mobile platforms. By integrating modern technologies such as Flask, PostgreSQL, Firebase, Kafka, Spark, Next.js, and React Native, the system ensures precise location reporting, real-time notifications, and efficient data management. Its modular architecture allows for easy maintenance and future expansion, while advanced features like map-based reporting, GPS-based mobile submissions, and filtering provide users and administrators with actionable insights. Overall, the project highlights how distributed, event-driven architectures can effectively support timely decision-making and improve situational awareness in a wide range of event reporting scenarios.

References

- [1] P. Pranay, S. V. Nanda Kishore, R. Rohith, R. Vijitha, R. Ala, and T. Zaid, "Neighbourhood Safety Alarm System," *International Journal on Science and Technology (IJSAT)*, vol. 16, no. 2, pp. 1–13, Apr.–Jun. 2025, doi: 10.71097/IJSAT.v16.i2.4365.
- [2] H. Zhang, R. Zhang, and J. Sun, "Developing real-time IoT-based public safety alert and emergency response systems," *Scientific Reports*, vol. 15, pp. 29056, Aug. 2025, doi: 10.1038/s41598-025-13465-7.
- [3] International Association of Fire Chiefs and PulsePoint Foundation, "Using Mobile Technology to Activate Citizens and First Responders," May 2018. [Online]. Available: https://www.iafc.org/docs/default-source/1assoc/white-paper-pulsepoint_final.pdf?sfvrsn=33b1890d_2. Last Accessed: 17 Dec. 2025.
- [4] Q. Tong, et al., "Mobile applications enhance out-of-hospital cardiac arrest outcomes: a systematic review and meta-analysis," *Scientific Reports*, vol. 15, pp. 1–10, Feb. 2025, doi: 10.1038/s41598-025-13465-7.
- [5] S. M. Shaffi, "Real-time incident reporting and intelligence framework: Data architecture strategies for secure and compliant decision support," *World Journal of Advanced Research and Reviews*, vol. 26, no. 3, pp. 110–118, Apr. 2025, doi: 10.30574/wjarr.2025.26.3.0849.
- [6] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," *Proceedings of the NetDB Workshop*, Athens, Greece, Jun. 2011.
- [7] M. Zaharia et al., "Apache Spark: A Unified Engine for Big Data Processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, Nov. 2016, doi: 10.1145/2934664.

- [8] T. Akidau et al., "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale Data Processing," *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1792–1803, Aug. 2015, doi: 10.14778/2824032.2824076.
- [9] J. Kreps, "Questioning the Lambda Architecture," *O'Reilly Radar*, Jul. 2014. [Online]. Available: <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>. Last Accessed: 17 Dec. 2025.
- [10] V. John and X. Liu, "A Survey of Distributed Message Broker Queues," *arXiv preprint arXiv:1704.00411*, Apr. 2017.
- [11] M. Zaharia et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 15–28, Apr. 2012.
- [12] Hashem, I. A. T., Yaqoob, I., Anuar, N. B., Mokhtar, S., Gani, A., and Khan, S. U., "The rise of big data on cloud computing: Review and open research issues", *Information Systems*, vol. 47, pp. 98–115, 2015. doi: 10.1016/j.is.2014.07.006.
- [13] Abdalla, H. B., "A brief survey on big data: technologies, terminologies and data-intensive applications", *Journal of Big Data*, vol. 9, art. 107, 2022, doi: 10.1186/s40537-022-00659-3.
- [14] Oussous, A., Benjelloun, F.-Z., Ait Lahcen, A., and Belfkih, S., "Big Data technologies: A survey.", *Journal of King Saud University – Computer and Information Sciences*, vol. 30, no. 4, pp. 431–448, 2018. doi: 10.1016/j.jksuci.2017.06.001.
- [15] Akidau, T., et al., "The Dataflow Model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing", *Proceedings of the VLDB Endowment (PVLDB)*, vol. 8, no. 12, pp. 1792–1803, 2015. doi: 10.14778/2824032.2824076.
- [16] Fielding, R. T., "Architectural Styles and the Design of Network-based Software Architectures", *Doctoral dissertation*, University of California, Irvine, 2000.
- [17] Kleppmann, M., "Designing Data-Intensive Applications", O'Reilly Media, 2017.
- [18] Kreps, J., Narkhede, N., and Rao, J., "Kafka: a Distributed Messaging System for Log Processing", *Proc. NetDB Workshop*, 2011.
- [19] M. Zook, M. Graham, C. Shelton, and S. Gorman, "Volunteered Geographic Information and Crowdsourcing in Disaster Response: A Case Study of the Haitian Earthquake," *World Medical & Health Policy*, vol. 2, no. 2, pp. 7–33, 2010.
- [20] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Real-time Data Systems*, Manning Publications, 2015.
- [21] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," in *Proceedings of the NetDB*, 2011.
- [22] M. Zaharia et al., "Spark: Cluster Computing with Working Sets," *USENIX HotCloud*, 2010.
- [23] A. Panyam, "High-Performance Geospatial Visualization with WebGL," *Journal of Web Engineering*, vol. 21, no. 4, 2022.